# Am I a Compiler Yet? A Benchmark for Feature Support in LLM-Based C-to-Rust Translation

Ariel E. Kellison, Laura Titolo, Ayon Basumallik, Niranjan Hasabnis, Cole Hunter, Chris Ladden, and Tom Yotam

Code Metal, Boston MA, USA
{ariel.kellison,laura,ayon,niranjan,cole,chris.ladden,tom}@codemetal.ai

**Abstract.** The migration of C codebases to Rust has gained significant traction, driven by the demand for stronger safety guarantees while maintaining performance. Large language models (LLMs) have shown promise for translating C programs to idiomatic Rust, but their support for individual C language features remains poorly understood. This paper introduces a compiler-inspired, feature-level validation benchmark for C-to-Rust translation that isolates individual C language constructs using targeted tests derived systematically from the C11 standard. Using this benchmark, we evaluate a wide range of LLMs and uncover substantial differences in their ability to translate individual C language features that may be hidden by existing whole-program benchmarks.

**Keywords:** C-to-Rust translation · language conformance testing · LLMs

## 1 Introduction

The translation of C programs to Rust is motivated by the need to evolve large, long-lived C codebases toward languages that provide stronger safety guarantees while preserving low-level control and performance. Existing C-to-Rust translation efforts have primarily relied on static analyses and rule-based rewriting [3–5,8,9,12,21]. These approaches offer predictability and explicit correctness criteria, but often suffer from limited coverage and difficulty producing idiomatic Rust. Recently, large language models (LLMs) have emerged as an alternative, demonstrating the ability to translate substantial portions of real-world C programs into readable Rust with minimal manual intervention [2,6,10,15,17–20,22].

However, unlike compiler-based translators, the behavior and limitations of LLM-based translation systems are not well characterized. While LLMs can succeed on many whole-program benchmarks, it remains unclear which parts of the C language they reliably support, and how robust that support is across syntactic and semantic variations. Moreover, because LLM-based translation does not proceed via explicit AST-to-AST rewriting or formal transformation rules, traditional compiler verification and validation techniques do not directly apply.

Program translation is inherently compositional: whole-program correctness depends on a correct handling of the individual language features exercised by

a program. In compiler validation, this observation motivates language conformance testing, in which implementations are evaluated against test suites derived from a language standard and designed to exercise individual language features.

This gap between whole-program success and feature-level support is especially pronounced for LLM-based translation systems, whose behavior can vary across model updates and decoding strategies. In industrial settings, where translation systems evolve rapidly, regressions in support for individual language features must be detected reliably. At Code Metal, we use feature-level validation for regression testing of LLM-based C-to-Rust translators, enabling fine-grained monitoring of translation behavior across system and model updates.

In this paper, we adapt the compiler-style validation perspective to the evaluation of LLM-based C-to-Rust translation. Rather than treating translation quality as a purely whole-program property, we evaluate translation behavior at the level of individual C language features, asking whether models provide systematic and reliable support for those features. This perspective complements existing whole-program evaluations and exposes failure modes that may remain hidden when multiple features interact in a larger program.

We present a validation approach for C-to-Rust translations that isolates and tests individual C language features using targeted, conformance-style programs derived from the C11 standard. Concretely, we make the following contributions.

**Contribution 1: A compiler-inspired validation methodology for LLM-based translation.** We introduce a language feature-level evaluation framework for C-to-Rust translation, inspired by traditional compiler conformance testing, which shifts the notion of correctness from whole-program behavior to the correct handling of individual language features.

**Contribution 2: A C11 language feature benchmark** (Section 2). We construct a benchmark of 131 tests derived from the C11 standard, isolating 46 core language features. Additionally, we introduce *feature coverage* and *translation consistency* to characterize LLM-based translation quality. The benchmark and evaluation framework are publicly available: https://github.com/CodeMetalAI/c-to-rust-language-feature-bench.

**Contribution 3: An empirical evaluation of state-of-the-art models** (Section 3). We evaluate a range of programming-specialized, flagship, and open general-purpose language models using the proposed benchmark. Our results show substantial variation in feature support and reliability among LLMs, even among models with similar aggregate translation scores.

## 2 Language Feature Support Benchmark

We introduce a feature-level benchmark for evaluating C-to-Rust translation systems. Inspired by compiler conformance testing, the benchmark isolates C language features using test cases derived from Clause 6 of the ISO/IEC 9899:2011 (C11) standard. We target C11 because it is widely supported by `gcc` and `clang` and extends C99, the focus of prior work on static C-to-Rust translation [11].

```c
goto lab3; /* invalid */
{
  double a[n];
  a[j] = 4.4;
lab3:
  a[j] = 3.3;
  goto lab4; /* valid */
  a[j] = 5.5;
lab4:
  a[j] = 6.6;
}
goto lab4; /* invalid */
```

```c
int main(void) {
  int n = 4, j = 1;
  int saw_lab3 = 0, saw_lab4 = 0;
  {
    double a[n];
    a[j] = 4.4;
    goto lab3; /* valid */
  lab3:
    saw_lab3 = 1;
    a[j] = 3.3;
    goto lab4; /* valid */
    a[j] = 5.5;
  lab4:
    saw_lab4 = 1;
    a[j] = 6.6;
  }
  if (!saw_lab3 || !saw_lab4)
    return 1;
  return 0;
}
```

Fig. 1: Original illustrative fragment from Example 2 in Clause 6.8.6.1 (left). Derived positive test case retaining only the well-formed `goto` behavior (right).

### 2.1 Constructing the Benchmark

The benchmark is constructed by extracting and adapting examples from the C11 standard into *positive test cases*, a methodology previously used in compiler conformance testing [7,14]. A positive test case is a program that is well-formed under the language rules and has fully-defined behavior; it compiles and executes successfully while exercising a specific language feature. In addition to illustrative examples, tests are also derived from prose descriptions in subclauses that do not contain examples but specify concrete, testable language rules.

Examples in the C11 standard are primarily explanatory and may include both well-formed and ill-formed code. The benchmark, however, consists exclusively of positive test cases derived from well-formed examples; when a standard example illustrates multiple behaviors of a feature, it may yield multiple tests. Figure 1 illustrates this extraction process using Example 2 from Clause 6.8.6.1, which concerns restrictions on `goto` statements involving variably modified types. The derived test retains only the valid `goto` behavior, ensuring that the resulting program is well-formed and exercises the intended feature.

The benchmark comprises 131 test cases spanning the C11 core language. Each test targets a single language feature, though multiple tests may capture distinct aspects of the same feature. Across the benchmark, the median C lines per test is 26; comments are excluded to avoid influencing model behavior. Tests are not uniformly distributed across features, reflecting the structure of the standard. Table 1 summarizes the resulting distribution[1].

A natural question is why existing C conformance or compiler validation test suites are not reused. Comprehensive suites such as those from Plum Hall [16] support feature-level validation but are proprietary and unsuitable for an

[1] Clauses 6.1, 6.6, and 6.11 are omitted because they contain no examples in the C11 standard, and we did not construct benchmark tests for them.

Table 1: **Benchmark coverage of C11 Clause 6.** For each subclause, we report the number of examples in the standard, the number of benchmark tests, and the number of distinct language features exercised by those tests.

| C11 Language Category | Examples | Tests | Features |
|---|---|---|---|
| §6.2 Concepts | 3 | 1 | 1 |
| §6.3 Conversions | 0 | 15 | 3 |
| §6.4 Lexical elements | 11 | 1 | 1 |
| §6.5 Expressions | 23 | 44 | 19 |
| §6.7 Declarations | 44 | 51 | 14 |
| §6.8 Statements | 7 | 7 | 4 |
| §6.9 External definitions | 4 | 4 | 2 |
| §6.10 Preprocessing directives | 13 | 8 | 2 |
| **Total** | **105** | **131** | **46** |

open benchmark. Public test suites from compilers such as `gcc` and `clang` are primarily designed for regression testing and contain relatively few systematic positive tests that isolate individual language features. We therefore construct an open benchmark derived directly from the C11 standard, preserving the spirit of conformance testing while remaining publicly available.

### 2.2 Evaluation Metrics

For each test, a translation is considered correct if the generated Rust code compiles and, when executed, exhibits the same observable behavior as the original C program. Equivalence is determined by comparing observable outputs such as return codes. The evaluation focuses on semantic translation correctness at the language feature level; we do not assess idiomatic quality, memory safety guarantees, or performance characteristics of the generated Rust code; unsafe Rust is permitted. A translation that fails to compile, panics at runtime, or diverges from the expected behavior is considered a failure. We define two metrics to characterize feature-level translation behavior.

*Feature Coverage.* Feature coverage measures whether a model successfully translates a given language feature. A feature is considered supported if the model produces a correct translation for at least one test exercising that feature.

*Translation Consistency.* Translation consistency measures how reliably a model translates a feature across benchmark instances. For a feature $f$ and model $m$, translation consistency is defined as the success rate:

$$SR(f, m) = \frac{\#\ \textit{tests of feature}\ f\ \textit{passed by}\ m}{\#\ \textit{tests of feature}\ f}. \tag{1}$$

## 3 Experimental Evaluation

We evaluate twelve large language models spanning flagship proprietary, open general-purpose, and programming-specialized models. Flagship models include GPT-4o, GPT-4-turbo, GPT-4o-mini, Claude Opus 4.5, and Claude Sonnet 4.5. Open general-purpose models include Gemma-2-9B, LLaMA-3.1 (8B and 70B), and DeepSeek-V3.2. Programming-specialized models include GPT-5.2-Codex, Grok-Code-Fast-1, and Mistral-Codestral-2508.

All models are evaluated under two decoding protocols. For the main barchart results in Figures 3 and 4, we use deterministic decoding (temperature 0) with a pass@3 metric. We additionally perform a temperature sweep with pass-rate analysis to characterize sensitivity to stochastic decoding. The maximum output length is set sufficiently high to avoid truncation of translated programs.

Each benchmark test is evaluated by prompting the model to translate the given C program into Rust. Translations are independent, without access to external tools or prior outputs. A translation is considered successful only if it compiles, executes with the expected observable behavior, and is not a semantically incomplete "stub". The prompt, compilation, and execution details, stub detection, and execution environment are described in Appendix A.

### 3.1 Model Performance Overview

We examine overall translation success across the full benchmark. Table 2 reports aggregate translation consistency under varying temperatures and pass@k settings. Our results show that performance varies widely across models, ranging from below 15% for smaller and lightweight models to above 80% for the strongest programming-specialized and flagship models. Increasing from pass@1 to pass@3 improves success for every model at every temperature (e.g., +0.7 to +17.6 pp under deterministic decoding and +3.1 to +22.9 pp at higher temperatures). The effect of temperature is model dependent: at pass@3, raising temperature from 0.0 to 0.8 improves 9/12 models (up to +13.8 pp for LLaMA-3.1-70B) but slightly degrades others, while at pass@1 the performance of most models decreases (8/12). These aggregate results reveal substantial variation in overall translation success but do not explain which C language features account for these differences, motivating a feature-level analysis in the following section.

### 3.2 Analysis of Language Feature Support

We now analyze translation behavior at the level of individual C language features using the feature coverage and translation consistency metrics defined in Section 2.2. Results in this section are reported under deterministic decoding (temperature 0) with pass@3, using three independent runs per model.

*Feature Coverage.* Figure 2 reports feature coverage for each model across the benchmark, where each row corresponds to an individual C language feature and rows are colored by their originating C11 clause. Coverage varies substantially

Table 2: Aggregate translation consistency across models at varying temperatures and pass rates.

| | **Temp = 0.0** | | **Temp = 0.2** | | **Temp = 0.8** | |
|---|---|---|---|---|---|---|
| **Model** | **Pass@1** | **Pass@3** | **Pass@1** | **Pass@3** | **Pass@1** | **Pass@3** |
| **Small Models** | | | | | | |
| Gemma 2 9B Instruct | 9.9% | 15.3% | 12.2% | 16.8% | 10.7% | **17.6**% |
| Llama 3.1 8B Instruct | 6.1% | 12.2% | 7.6% | 13.7% | 4.6% | 12.2% |
| **Large Models** | | | | | | |
| Llama 3.1 70B | 28.2% | 32.8% | 27.4% | 38.9% | 27.4% | 46.6% |
| DeepSeek V3.2 | 54.2% | 71.8% | 51.9% | 72.5% | 58.0% | **76.3**% |
| **Programming-Specialized Models** | | | | | | |
| Grok Code Fast 1 | 65.6% | 81.7% | 61.8% | 84.7% | 58.0% | 80.9% |
| Mistral Codestral 2508 | 49.6% | 59.5% | 48.0% | 55.0% | 45.0% | 66.4% |
| GPT-5.2-codex | 84.7% | **91.6%** | 86.3% | 89.3% | 87.0% | 90.8% |
| **Flagship Models** | | | | | | |
| GPT-4o | 46.6% | 58.8% | 50.4% | 57.2% | 45.0% | 61.8% |
| GPT-4o turbo | 58.0% | 63.3% | 59.5% | 64.9% | 51.1% | 72.5% |
| Claude Opus 4.5 | 81.7% | 82.4% | 78.6% | 82.4% | 80.9% | **84.0%** |
| Claude Sonnet 4.5 | 75.6% | 78.6% | 74.0% | 81.7% | 74.8% | 79.3% |
| **Lightweight Flagship Models** | | | | | | |
| GPT-4o mini | 9.9% | 10.7% | 9.2% | 10.7% | 9.9% | 17.6% |

across features: although a majority of features (**32 of 46**) are supported by more than half of the models, universal support is rare, with only **3 of 46** features covered by all models. Moreover, a substantial minority of features (**11 of 46**) are supported by six or fewer of the twelve models.

These results indicate that language feature support is broad but uneven, and that aggregate translation success can obscure systematic gaps in feature-level support. Notably, for every feature in the benchmark, at least one model achieves coverage, demonstrating that correct Rust implementations exist for the benchmark instances considered. Failures observed for other models therefore reflect limitations in model behavior rather than inherent barriers to translation.

*Translation Consistency.* We next examine translation consistency across the benchmark. Figure 3 reports per-test success rates—computed using Equation (1)—for tests targeting declaration features from §6.7 of the C11 standard, with separate plots for programming-specialized, flagship, and open general-purpose models; results for expression features from §6.5 appear in Figure 4 of Sec-

tion B.1. Across both clauses, consistency varies markedly by model category: programming-specialized models exhibit relatively stable performance, whereas open general-purpose models show substantially greater instance-level variability, with interleaved successes and failures within the same clause. Flagship models fall between these extremes, achieving higher overall consistency than open models but still exhibiting pronounced instance-level variation.

For the remaining clauses—concepts, conversions, statements, lexical elements, external definitions, and preprocessing directives—we observe similar but coarser trends. These clauses contain fewer benchmark instances, and their translation consistency largely mirrors the patterns seen in expressions and declarations: programming-specialized models are more reliable, whereas open general-purpose models exhibit greater variability. As a result, we omit per-instance plots for these clauses and summarize their behavior at the feature level.

In addition, for the programming-specialized models, a small number of features are translated perfectly across all benchmark instances and runs. Specifically, six expression features and three declaration features achieve a success rate of 1.0 for all programming-specialized models and are therefore omitted from Figures 3a and 4a to improve readability.

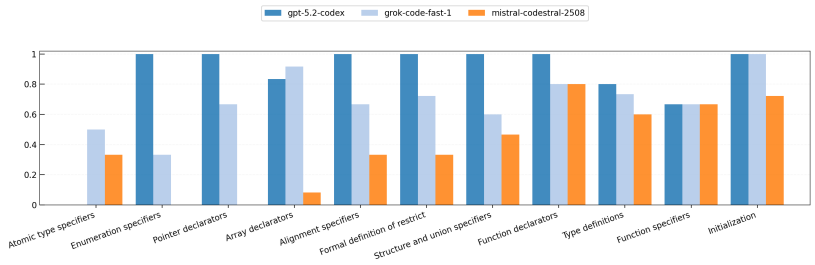
## 4 Related Work

Prior work on C-to-Rust translation evaluates systems using whole-program benchmarks. `c2rust` evaluates translation success on a set of 11 C programs ranging in size from 27 to 201,695 lines of code, with a median project size of 776 lines, using compilation, testing, and execution cross-checking to validate semantic preservation. Follow-on systems such as Laertes [5] and Crown [21] focus on generating safe Rust and adopt similar evaluation methodologies, reusing the `c2rust` project-level benchmarks and extending it with additional programs. Laertes reports results on 17 programs with a median size of 5,002 lines of code, while Crown evaluates on 20 programs with a median size of 2,510 lines of code.

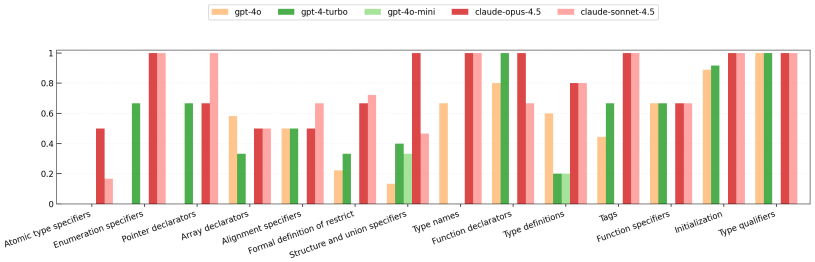
Unlike LLM-based translators, these static systems typically make explicit claims about their intended scope and limitations with respect to the C language. `c2rust` targets C99-compliant code and documents unsupported features such as variadic functions, `setjmp`/`longjmp`, inline assembly, `restrict` pointers, SIMD intrinsics without Rust equivalents, and certain floating-point extensions. Successor systems such as Laertes and Crown inherit these limitations and further refine the translation pipeline, focusing on reducing specific classes of unsafety in the generated Rust rather than expanding the supported source-language fragment. In this sense, Laertes and Crown are best viewed as analysis and rewrite-based extensions of `c2rust`, rather than independent translators.

While these systems clearly articulate which C language features are supported, their evaluations do not systematically test that support at the level of individual language features. Feature support is instead inferred indirectly from whole-program success on a small set of benchmarks. This makes it difficult to

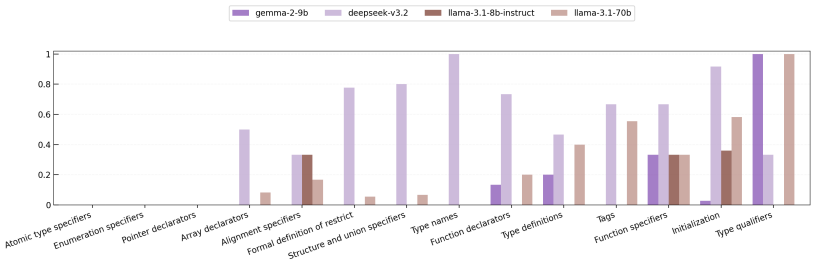
Fig. 2: **Feature coverage by model and feature category.** Each colored cell indicates whether a model achieves feature coverage for a language feature, defined as successfully translating at least one test exercising that feature. Cell color denotes the feature category.




(a)




(b)




(c)

Fig. 3: Translation success rates for (a) programming-specialized, (b) flagship, and (c) open general-purpose models for features in the **declarations** category.

determine whether a feature is handled correctly in general, or only in restricted or favorable usage patterns present in the evaluated programs.

The benchmark introduced in this paper is complementary to these existing evaluation practices. By isolating individual C language features and testing them using small, conformance-style programs derived from the C standard, our benchmark enables direct validation of claimed feature support. Applied to static C-to-Rust translators, it could be used to empirically assess conformance with the intended source-language subset (e.g., C99), distinguish robust feature support from partial or brittle implementations, and provide coverage-style accounting of which language rules are exercised by existing benchmarks. While our experimental evaluation focuses on LLM-based translators, the benchmark itself is tool-agnostic and applicable to both static and learned translation systems.

Recent work on LLM-based C-to-Rust translation [2,6,10,15,17–20,22] adopts a similar project-level evaluation paradigm, emphasizing end-to-end translation quality and idiomatic target-language output. CRUST-Bench [13] formalizes this approach by introducing a repository-scale benchmark with annotated Rust interfaces and correctness-enforcing test cases, enabling systematic end-to-end evaluation of LLM-based translators. While these studies demonstrate that LLMs can translate nontrivial programs, they provide limited evidence about systematic support for individual C language features. This motivates the use of feature-isolating, standard-derived benchmarks as a complementary evaluation method for both static and LLM-based translators.

## 5 Conclusion and Future Work

This paper presented a feature-level validation framework for evaluating C-to-Rust code translation systems. Inspired by compiler conformance testing, the framework treats translation correctness as a property of individual language features rather than aggregate program success. It introduces a benchmark derived from the C11 standard that isolates core language features, together with feature coverage and translation consistency metrics for systematic assessment.

Using this benchmark, we evaluated a range of state-of-the-art large language models across multiple model categories. Feature support is broad but uneven: while many features are handled by most models, universal support is rare, and a substantial fraction of features are supported by only a few models. Moreover, even supported features exhibit significant instance-level variability, with pronounced differences between programming-specialized, flagship, and open general-purpose models. These results mirror traditional compiler testing, where success on many programs does not imply full language conformance, and expose limitations of whole-program evaluations for LLM-based translation.

This framework can be extended to assess additional dimensions of translation quality, including memory safety and other Rust-specific correctness properties. We aim to combine the benchmark with program fuzzing and constrained LLM generation techniques [1] to generate larger and more diverse test suites, further stressing translation consistency and robustness.

## References


1. Anaya Gonzalez, E., Vaidya, S., Park, K., Ji, R., Berg-Kirkpatrick, T., D'Antoni, L.: Constrained sampling for language models should be easy: an MCMC perspective. CoRR **abs/2506.05754** (2025). https://doi.org/10.48550/ARXIV.2506.05754, https://doi.org/10.48550/arXiv.2506.05754
2. Cai, X., Liu, J., Huang, X., Yu, Y., Wu, H., Li, C., Wang, B., Yusuf, I.N.B., Jiang, L.: RustMap: Towards Project-Scale C-to-Rust Migration via Program Analysis and LLM. In: Engineering of Complex Computer Systems: 29th International Conference, ICECCS 2025, Hangzhou, China, July 2-4, 2025, Proceedings. pp. 283–302. Springer-Verlag, Berlin, Heidelberg (2025). https://doi.org/10.1007/978-3-032-00828-2_16, https://doi.org/10.1007/978-3-032-00828-2_16
3. Citrus Contributors: Citrus. https://gitlab.com/citrus-rs/citrus (2017), accessed 2017
4. Corrode Contributors: Corrode. https://github.com/jameysharp/corrode (2017), accessed 2017
5. Emre, M., Schroeder, R., Dewey, K., Hardekopf, B.: Translating C to safer Rust. Proc. ACM Program. Lang. **5**(OOPSLA) (Oct 2021). https://doi.org/10.1145/3485498, https://doi.org/10.1145/3485498
6. Ferit Eniser, H., Zhang, H., David, C., Wang, M., Christakis, M., Paulsen, B., Dodds, J., Kroening, D.: Towards translating real-world code with LLMs: a study of translating to Rust (2025), https://arxiv.org/abs/2405.11514
7. Gibbs, T.H., Malloy, B.A., Power, J.F.: Progression toward conformance of C++ language compilers. Dr. Dobb's Journal **28**(11), 54–60 (2003)
8. Hong, J., Ryu, S.: Don't Write, but Return: Replacing Output Parameters with Algebraic Data Types in C-to-Rust Translation. Proc. ACM Program. Lang. **8**(PLDI) (Jun 2024). https://doi.org/10.1145/3656406, https://doi.org/10.1145/3656406
9. Hong, J., Ryu, S.: To Tag, or Not to Tag: Translating C's Unions to Rust's Tagged Unions. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. pp. 40–52. ASE '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3691620.3694985, https://doi.org/10.1145/3691620.3694985
10. Hong, J., Ryu, S.: Type-migrating C-to-Rust translation using a large language model. Empirical Software Engineering **30**(1), 3 (2024). https://doi.org/10.1007/s10664-024-10573-2, https://doi.org/10.1007/s10664-024-10573-2
11. Immunant, Inc.: C2Rust manual. https://c2rust.com/manual/ (2020), accessed 2025-11-04
12. Immunant, Inc.: immunant/c2rust. https://github.com/immunant/c2rust (2020), gitHub repository. Accessed 2025-11-04
13. Khatry, A., Zhang, R., Pan, J., Wang, Z., Chen, Q., Durrett, G., Dillig, I.: CRUST-Bench: a comprehensive benchmark for C-to-safe-Rust transpilation. In: Second Conference on Language Modeling (2025), https://openreview.net/forum?id=8xofWL61S9
14. Malloy, B.A., Linde, S.A., Duffy, E.B., Power, J.F.: Testing C++ compilers for ISO language conformance. Dr. Dobb's Journal **27**(6), 71–78 (2002)
15. Nitin, V., Krishna, R., Lemos do Valle, L., Ray, B.: C2SaferRust: Transforming C Projects into Safer Rust with NeuroSymbolic Techniques (2025), https://arxiv.org/abs/2501.14257
16. Plum Hall Inc.: The Plum Hall Validation Suite. https://www.plumhall.com/newsite/stec1.html, accessed January 2026

17. Shetty, M., Jain, N., Godbole, A., Seshia, S.A., Sen, K.: Syzygy: Dual Code-Test C to (safe) Rust Translation using LLMs and Dynamic Analysis (2024), https://arxiv.org/abs/2412.14234
18. Shiraishi, M., Shinagawa, T.: Context-aware Code Segmentation for C-to-Rust Translation using Large Language Models (2024), https://arxiv.org/abs/2409.10506
19. Tang, Z., Agarwal, M., Shypula, A., Wang, B., Wijaya, D., Chen, J., Kim, Y.: Explain-then-translate: an analysis on improving program translation with self-generated explanations. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023. pp. 1741–1788. Association for Computational Linguistics, Singapore (Dec 2023). https://doi.org/10.18653/v1/2023.findings-emnlp.119, https://aclanthology.org/2023.findings-emnlp.119/
20. Yang, A.Z.H., Takashima, Y., Paulsen, B., Dodds, J., Kroening, D.: Vert: Verified equivalent Rust transpilation with large language models as few-shot learners (2024), https://arxiv.org/abs/2404.18852
21. Zhang, H., David, C., Yu, Y., Wang, M.: Ownership Guided C to Rust Translation. In: Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part III. pp. 459–482. Springer-Verlag, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-031-37709-9_22, https://doi.org/10.1007/978-3-031-37709-9_22
22. Zhou, T., Lin, H., Jha, S., Christodorescu, M., Levchenko, K., Chandrasekaran, V.: LLM-driven multi-step translation from C to Rust using static analysis. In: NeurIPS 2025 Fourth Workshop on Deep Learning for Code (2025), https://openreview.net/forum?id=i5MwgkeGZN

# A Experimental Details

This appendix documents the experimental setup used to evaluate C-to-Rust translations, including prompting, compilation and execution criteria, stub detection, and the execution environment.

## A.1 Prompt Format

```
Convert the following C++ program to an equivalent standalone
Rust program.

Rules:
- Preserve behavior exactly (same stdout and exit code).
- Must compile with rustc.
- Use idiomatic Rust types/constructs where possible.

Output:
Return ONLY the Rust code in a single ‘‘‘rust’’’ code block,
with a ‘fn main()’ and no other text.
```

## A.2 Compilation and Execution

Generated Rust programs are compiled using `rustc` with default settings. A translation is considered successful if the program compiles and, when executed, produces the expected observable behavior of the original C program, as defined in section 2.2.

Details of the compilation toolchains and execution platform used in the experiments described in this paper are provided in Section A.4.

## A.3 Stub Detection

In addition to compilation and execution checks, we apply a stub-detection pass to filter translations that are syntactically valid but semantically incomplete. The detector compares each generated Rust program against a structural baseline derived from static analysis of the original C code, capturing properties such as program size, function definitions, control flow, and nontrivial function calls. It also checks for common stub patterns, including literal-only returns, print-only programs, and explicit placeholder markers, and verifies symbol coverage when available. Translations flagged as stubs are treated as failed attempts and are not counted as successful translations, even if they compile and run.

## A.4 Environment Details

All experiments were conducted on macOS 15.7.3 running on an arm64 system (Apple M4 Max). C programs were compiled using Apple Clang 17.0.0 via the `gcc` driver, and generated Rust programs were compiled and executed using `rustc` 1.91.0-nightly (7ad23f43a, 2025-09-09).

# B Additional Experimental Evaluation

This appendix provides additional experimental results extending the main evaluation in Section 3, including per-test translation consistency results for expression features from §6.5 of the C11 standard and a detailed breakdown of aggregate outcomes by compilation and stub detection.

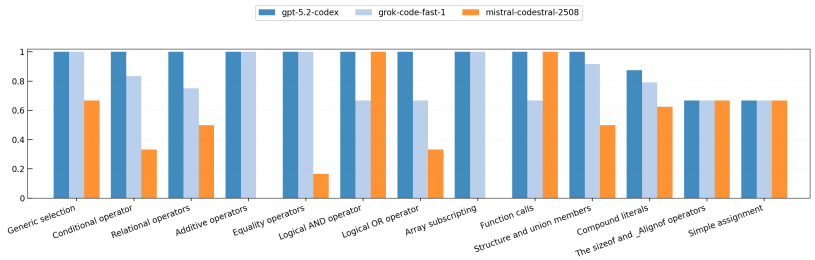
## B.1 Translation Consistency for Expressions

Per-test translation consistency results for benchmark tests targeting expression features from §6.5 of the C11 standard are shown in Figure 4; their interpretation is discussed in the translation consistency paragraph of Section 3.

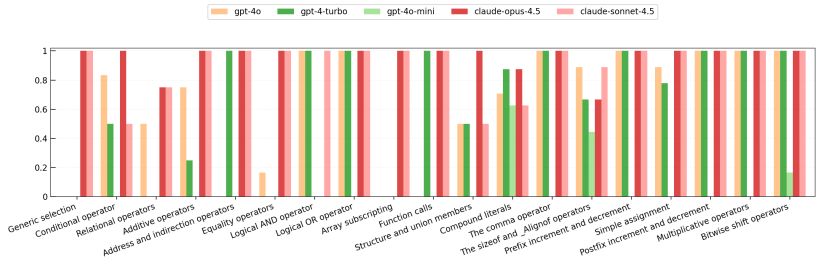
Each bar corresponds to a single benchmark test, with height indicating the fraction of runs in which the model produced a correct translation. Plots are grouped by model category to illustrate instance-level variability within each group.

## B.2 Outcome Decomposition and Stub Detection

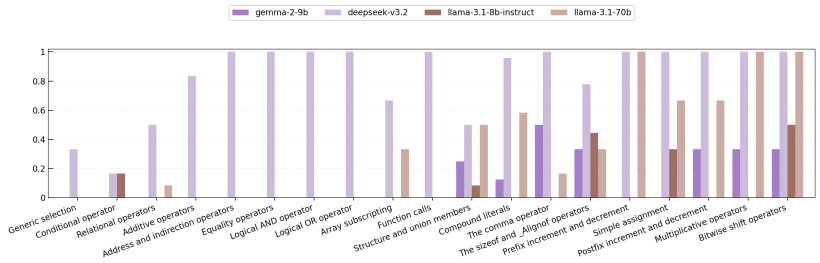
This section extends the aggregate performance analysis in Section 3.1 by reporting additional outcome metrics under varying temperatures and sampling budgets, alongside the overall translation success rates reported in Table 2. These results are summarized in Table 3. The rate at which generated Rust code compiles (**CR**) measures the fraction of benchmark tests for which the generated Rust program compiles successfully, while the stub-detection rate (**SD**) measures the fraction of tests for which the generated program is classified as a syntactically valid but semantically incomplete stub according to the criteria in Section A.3. Together with the overall translation success rate (**SR**), these metrics decompose unsuccessful translations into failures due to invalid Rust code, stub outputs, and remaining semantic mismatches among compiling, non-stub programs, as evidenced by incorrect observable behavior at execution time.




(a)




(b)




(c)

Fig. 4: Translation success rates for (a) programming-specialized, (b) flagship, and (c) open general-purpose models for features in the **expressions** category.

Table 3: Per-model performance by temperature and sampling budget. For each temperature and Pass@$k$, we report the overall translation success rate (**SR**), the rate at which generated Rust code compiles (**CR**), and the rate at which translations are identified as stubs (**SD**).

| **Model** | **Temp = 0.0** | | | | | | **Temp = 0.2** | | | | | | **Temp = 0.8** | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | **Pass@1** | | | **Pass@3** | | | **Pass@1** | | | **Pass@3** | | | **Pass@1** | | | **Pass@3** | | |
| | **SR** | **CR** | **SD** | **SR** | **CR** | **SD** | **SR** | **CR** | **SD** | **SR** | **CR** | **SD** | **SR** | **CR** | **SD** | **SR** | **CR** | **SD** |
| **Small Models** | | | | | | | | | | | | | | | | | | |
| Gemma 2 9B Instruct | 9.9% | 16.8% | 7.6% | 15.3% | 21.0% | 5.3% | 12.2% | 16.0% | 8.4% | 16.8% | 22.1% | 5.3% | 10.7% | 18.3% | 9.2% | **17.6%** | 26.0% | 4.6% |
| Llama 3.1 8B Instruct | 6.1% | 9.1% | 1.5% | 12.2% | 17.6% | 0.8% | 7.6% | 8.4% | 4.6% | 13.7% | 17.6% | 0.8% | 4.6% | 11.5% | 2.3% | 12.2% | 22.9% | 0% |
| **Large Models** | | | | | | | | | | | | | | | | | | |
| Llama 3.1 70B | 28.2% | 35.1% | 3.1% | 32.8% | 45.0% | 3.1% | 27.4% | 33.6% | 4.6% | 38.9% | 48.1% | 1.5% | 27.4% | 31.2% | 4.6% | 46.6% | 58.0% | 1.5% |
| DeepSeek V3.2 | 54.2% | 61.8% | 2.3% | 71.8% | 79.4% | 0.8% | 51.9% | 61.0% | 2.3% | 72.5% | 79.4% | 0.8% | 58.0% | 67.2% | 3.1% | **76.3%** | 83.2% | 0% |
| **Programming-Specialized Models** | | | | | | | | | | | | | | | | | | |
| Grok Code Fast 1 | 65.6% | 73.3% | 0.8% | 81.7% | 93.9% | 0.8% | 61.8% | 71.0% | 2.3% | 84.7% | 94.7% | 1.5% | 58.0% | 68.7% | 2.3% | 80.9% | 92.4% | 0.8% |
| Mistral Codestral 2508 | 49.6% | 62.5% | 1.5% | 59.5% | 73.3% | 0.8% | 48.0% | 60.3% | 1.5% | 55.0% | 67.9% | 0.8% | 45.0% | 59.5% | 1.5% | 66.4% | 77.9% | 0% |
| GPT-5.2-codex | 84.7% | 91.6% | 0.8% | **91.6%** | 97.7.0% | 0.8% | 86.3% | 93.1% | 1.5% | 89.3% | 97.7% | 0.8% | 87.0% | 93.1% | 0.8% | 90.8% | 96.9% | 0.8% |
| **Flagship Models** | | | | | | | | | | | | | | | | | | |
| GPT-4o | 46.6% | 53.4% | 0.8% | 58.8% | 65.6% | 0% | 50.4% | 58.0% | 0.8% | 57.2% | 64.9% | 1.5% | 45.0% | 51.9% | 0.8% | 61.8% | 83.2% | 0% |
| GPT-4o turbo | 58.0% | 69.5% | 3.0% | 63.3% | 76.3% | 2.3% | 59.5% | 68.7% | 3.1% | 64.9% | 75.6% | 3.0% | 51.1% | 64.9% | 5.3% | 72.5% | 83.2% | 0.8% |
| Claude Opus 4.5 | 81.7% | 96.2% | 6.1% | 82.4% | 97.7% | 6.1% | 78.6% | 93.9% | 6.1% | 82.4% | 97.7% | 6.1% | 80.9% | 97.7% | 6.1% | **84.0%** | 98.5% | 5.3% |
| Claude Sonnet 4.5 | 75.6% | 84.0% | 1.5% | 78.6% | 87.0% | 1.5% | 74.0% | 83.2% | 1.5% | 81.7% | 88.5% | 0.8% | 74.8% | 84.0% | 0.8% | 79.3% | 87.0% | 0.8% |
| **Lightweight Flagship Models** | | | | | | | | | | | | | | | | | | |
| GPT-4o mini | 9.9% | 11.5% | 1.5% | 10.7% | 13.0% | 0% | 9.2% | 11.5% | 1.5% | 10.7% | 13.7% | 0% | 9.9% | 12.2% | 0.8% | 17.6% | 21.4% | 0% |